

SmartKasi: what is built, and what stands between it and a pilot

The backend is real and running. The three Flutter apps are real. What does not yet exist is everything that turns a seeded demo into something a shop owner in Soweto could sign up for on their own.

API `api-production-5594.up.railway.app/v1` — 200 OK
DB Supabase `wndilblmkkdypffmwap`, eu-west-1 Branch `feat/real-delivery`
Routes 45 across 14 modules

The one-line read

The engineering is better than the documentation admits in some places and worse in others — but the gap that matters is not a feature gap. **Three defects mean that any user who is not already a row in `db/seed.sql` cannot use two of the three apps at all**, and the shop owner app cannot do the one thing the whole architecture was designed around.

Everything in the docs marked “verified” is verified against five hand-provisioned demo users. That was the right call for a Saturday deadline. It is now the thing hiding the real backlog: **role assignment, offline catalogue, and an operator console — none of which exist**, and all three of which are pilot-blocking rather than nice-to-have.

State of play

What is actually in the repository today, ignoring what the READMEs claim.

LIVE API — NestJS + Prisma 45 routes, 14 modules, deployed. Health check returned 200 today.	LIVE Postgres schema 16 tables, 11 enums, 8 triggers, 10 RLS policies. Stock as a ledger, not a counter.	LIVE OpenAPI contract 2,653 lines, frozen, additive-only. The strongest asset on the project.
---	---	--

LIVE Courier dispatch Request → job board → accept → collect → handover, with a race-safe accept.	LIVE Multi-shop orders Quote, place, per-shop accept/reject legs, recalculated totals, partial acceptance.	PARTIAL Customer app Browse, compare, cart, quote, order, track. Sits on the shared Flutter package.
PARTIAL Courier app Jobs, active delivery, payout view. No way to register as a courier.	PARTIAL Shop owner app Dashboard, POS, inventory, order queue. The offline claim is not met — see SK-02.	NOT STARTED Web dashboard Four files of unmodified create-next-app. Zero product code, despite the README.
NOT STARTED Operator / admin surface Licences can be submitted and never approved. Couriers can never be verified.	STUB AI dish → ingredients Returns the same pap-and-chakalaka basket for every input. Shape is right, no model call.	STUB Payments Returns <code>not_implemented</code> on purpose. v1 is cash — a decision, not a gap.
NOT STARTED Automated tests Zero <code>.specs</code> in the API. One 38-line Flutter test. Smoke scripts only.	NOT STARTED CI No <code>.github/</code> . Nothing runs on push or on a pull request.	

Core milestones

Six, in dependency order. The numbering is a sequence, not a ranking — M1 and M2 gate any pilot, and M3 gates the pilot growing past the shops you onboard by hand.

M1

PILOT BLOCKER

BACKEND + DB

A stranger can sign up and become a shop owner or a courier

Today the role that authorises every request comes from the JWT's `app_metadata.role`, and the only thing that ever writes it is a service-role script the team runs by hand. The schema documents a mirroring trigger that was never written. A real signup lands as `customer` permanently, so the courier app and the owner app have no reachable front door. This is the single largest blocker on the project and it is roughly a day of work.

DONE WHEN

A brand-new email signs up in the courier app, is verified, and picks up a job — with no one touching the database, and no service-role key anywhere near a phone.

Closes [SK-01](#), [SK-05](#), [SK-06](#)

M2

PILOT BLOCKER

FLUTTER

The till keeps selling when the network drops

The offline story is the project's headline differentiator and its cleanest piece of backend design — one unique constraint on `(shop_id, client_sale_id)`, no sync engine. The client never took delivery of it. Barcode lookup goes over the network on every scan, so with no signal the scan fails, the cart stays empty and there is no sale to queue. GET `/shops/{id}/sync` is live on the API and is not called from anywhere in the Flutter codebase.

DONE WHEN

Aeroplane mode: scan five items from the local catalogue, take cash, print change, close the app, reopen it, restore signal — and the sales flush themselves without anyone pressing a button.

Closes [SK-02](#), [SK-03](#), [SK-04](#)

M3

GROWTH BLOCKER

NEXT.JS

Someone can operate the platform without SQL

The dashboard is an empty Next.js scaffold. Every operational action — approving a shop licence, verifying a courier, unsticking an order, answering “where is my delivery” — currently requires a developer with a service-role key. That caps the pilot at however many shops the team can babysit personally. Scope it small: four screens, read-mostly, two write actions.

DONE WHEN

A non-developer approves a licence, verifies a courier, and finds a stuck order by phone number, in the browser.

Closes [SK-07](#), [SK-08](#), [SK-09](#)

M4

AFTER M1-M3

BACKEND

It survives a restart, a redeploy and a bad afternoon

Quotes live in a process-local Map, so a restart or a second instance empties a customer's checkout mid-basket. There are no tests behind any of the money paths — the stock ledger, the idempotent batch replay, the fee calculation — and no CI, so the first regression will be found by a shop owner. The smoke suite is good but it writes to the database it tests, which makes it unusable against anything but demo data.

DONE WHEN

Quotes survive a redeploy, a push runs lint plus tests plus a contract lint, and the money paths have tests that do not need a live database.

Closes [SK-10](#), [SK-11](#), [SK-12](#), [SK-13](#)

M5

DECISION, NOT CODE

PRODUCT

The commercial numbers are decided by a person, not a default

R10 base, R5 per extra shop, R1.50/km, 80% courier share, 2 km maximum basket spread — every one of these is a placeholder that has been open since the brief, and every one is now baked into a live API that three client apps display to users. The courier share in particular is a unit-economics decision: at 80% of a R10 fee the platform earns R2 per delivery, which does not fund the platform. Cheap to change, expensive to have wrong in front of a pilot shop.

DONE WHEN

The five constants are signed off with a stated rationale and moved out of “open questions” in the contract doc.

Closes [SK-14](#)

The dish basket actually looks at the dish

Deliberately last. The stub's response shape is already correct and drops straight into `POST /orders/quote`, so the client work is done and the swap is contained to one controller. It is the most demo-able feature on the project and the least load-bearing — shipping it before M1 and M2 would be optimising the pitch over the product.

DONE WHEN

Three dishes nobody hard-coded return sensible baskets priced from real nearby inventory, with a fallback that degrades to search when the model is unavailable.

Closes [SK-15](#)

Issue register

Fifteen open items, each traced to the file that proves it. P0 blocks a pilot, P1 blocks scale or is a real production risk, P2 is deferred debt with a decision attached.

ID	SEV	ISSUE	EVIDENCE	MS
SK-01	P0	The role-mirroring trigger does not exist The schema comment promises that <code>profiles.role</code> is mirrored into <code>raw_app_meta_data</code> by a trigger. No such trigger was ever written. The guard silently defaults to <code>customer</code> , so authorisation works only for users created by the seeding script.	<code>db/schema.sql:76</code> promises it; <code>db/schema.sql:475-490</code> does not do it; <code>supabase-auth.guard.ts</code> <code>toUser()</code> defaults; only <code>scripts/seed-users.mjs:53</code> sets it	M1
SK-02	P0	Offline POS cannot look up a product offline Every scan calls the API. With no signal the lookup throws before anything reaches the cart, so the offline queue only catches a network drop in the seconds between building a basket and taking cash — not the case it was built for.	<code>shop_owner/pos.dart:35-52</code> calls <code>api.barcodeLookup</code> ; no local catalogue store exists	M2
SK-03	P0	The delta-sync endpoint is live and unused <code>GET /shops/{id}/sync</code> is implemented, documented and tested on the server. There is no corresponding method in the Flutter API client, so the mechanism the offline design depends on has no consumer.	<code>sync/sync.controller.ts:15</code> exists; <code>smartkasi_shared/lib/src/api.dart</code> has no sync call	M2
SK-04	P1	Queued sales never flush themselves <code>connectivity_plus</code> is a declared dependency and is never imported anywhere. Flushing is a manual button, so a day of cash sales sits in <code>shared_preferences</code> as plaintext JSON until someone remembers — and is lost if the app data is cleared.	<code>pubspec.yaml</code> declares it, zero imports; <code>controllers.dart:224-265</code> flushes only on demand	M2

ID	SEV	ISSUE	EVIDENCE	MS
SK-05	P0	No courier onboarding path The couriers table carries mode, radius, vehicle registration, ID document and a verification flag. No endpoint creates or updates a row, and none toggles is_online. A courier can only exist if someone inserts them.	db/schema.sql:360-370 ; no route under /courier beyond jobs, accept, collect, deliver	M1
SK-06	P1	Shop licences can be submitted but never approved POST /shops/{id}/licence accepts a submission. Nothing in the API ever moves a shop to verified, so an unverified shop is permanently unable to take orders.	shops.controller.ts:38 ; no approval route anywhere	M1
SK-07	P1	The web dashboard is unmodified boilerplate Four files of default create-next-app, while the root README describes it as the admin dashboard. Anyone planning against that README is planning against something that does not exist.	apps/web/src/app/ — layout, page, globals.css, favicon	M3
SK-08	P1	No operator view of orders or deliveries There is no way to answer a support call. Order and delivery state can only be read by the customer who owns it, the shop on the leg, or a developer with database access.	Ownership checks in orders.service.ts:132 , delivery.service.ts:79,131 ; no operator surface	M3
SK-09	P2	Flyers and image uploads are live with no consumer Three flyer routes and an R2 presign endpoint are implemented and called by nothing. R2 credentials are blank in the environment template, so the presign path has almost certainly never run.	flyers.controller.ts , uploads.controller.ts:8 ; no Flutter or web caller; .env.example R2 keys empty	M3
SK-10	P1	Quotes are held in process memory A restart, a redeploy or a second instance produces a spurious expiry mid-checkout. Known and documented in the code — it becomes a real incident the moment there is more than one dyno or one deploy during trading hours.	orders/quote.service.ts — private readonly quotes = new Map()	M4
SK-11	P1	No automated tests on any money path Zero test files in the API. The stock ledger, the idempotent batch replay, the fee calculation and the leg-acceptance state machine all move money or inventory and none has a test.	No *.spec.ts under apps/api/src ; one 38-line Flutter test	M4
SK-12	P1	No CI No .github/ directory. Nothing runs on push: not the type check, not the linter, not the contract lint that protects the frozen OpenAPI file.	Repository root has no CI configuration	M4
SK-13	P2	No request IDs, structured logs or rate limiting Every “it does not work” message during the pilot will be unanswerable without a correlatable log line. Rate limiting is fifteen minutes of work on a public API.	Default Nest logger; no throttler dependency in apps/api/package.json	M4
SK-14	P1	Fee constants and basket spread are still placeholders Five numbers with real commercial consequences are shipping as environment defaults with “placeholder” written next to them in the code. The courier share of 80% leaves roughly R2 per delivery for the platform.	config/configuration.ts fees block; open questions 1 and 3 in docs/API_CONTRACT.md	M5

ID	SEV	ISSUE	EVIDENCE	MS
SK-15	P2	AI dish endpoint returns a fixed basket Correctly built as an honest stub with a final response shape and a <code>_stub</code> flag. Listed so it is tracked, not because it should be done soon.	<code>stubs/ai.controller.ts:22</code>	M6

Decisions needed before M1 starts

Each of these changes what gets built. None of them is a developer's call.

PRODUCT

How does someone become a shop owner?

Self-declare at signup and get verified later, or apply and wait for approval? The first ships in a day and lets anyone claim a shop; the second needs M3 to exist first. This choice sets the shape of M1.

PRODUCT · FINANCE

What does the platform earn per delivery?

80% of the service fee to the courier leaves about R2 on a single-shop, short-distance order. Either the fee goes up, the share comes down, or the delivery business is a customer-acquisition cost with a stated budget.

PRODUCT

Is a courier a person you recruit or a person who signs up?

The build order already flags that courier supply is a business problem, not a sprint. If couriers are recruited and vetted by hand for the pilot, SK-05 shrinks to a form in the dashboard and M1 gets smaller.

ENGINEERING · PRODUCT

Do we hold the offline claim in the pitch?

Until M2 lands, “offline-capable POS” is not true in the app. Either M2 goes first, or the claim comes out of the deck — demonstrating it to a shop owner who then loses signal is worse than not claiming it.

Assessment based on the `feat/real-delivery` branch at commit 21b3eab, with the deployed API checked live. Endpoint counts are from the controller decorators; the LIVE/STUB claims in `docs/API_CONTRACT.md` were re-checked against the source rather than taken at face value.